

1. Componentes (La Vista)

¿Qué son?

Los componentes son las **piezas visuales** de tu aplicación. Cada uno tiene una **responsabilidad única** y se encarga de **mostrar** información y **capturar** interacciones del usuario.

¿Cuántos componentes tienes?

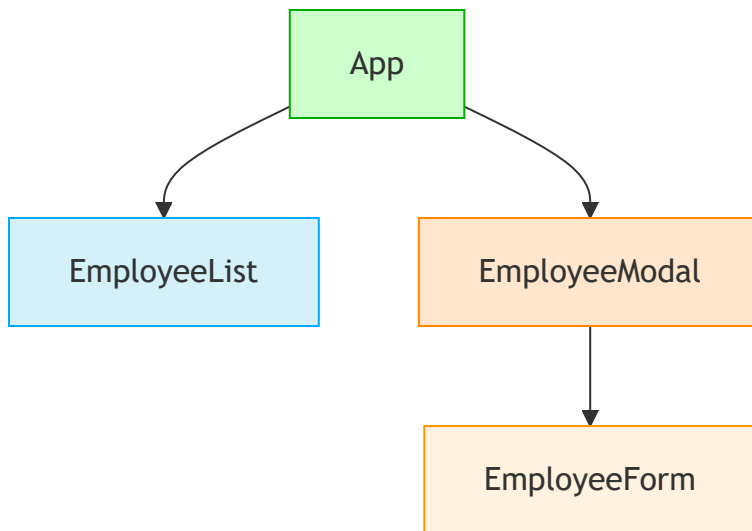
Componente	Responsabilidad	¿Qué muestra?	¿Qué recibe?
App	Orquestador principal	Navbar + Lista + Modal	Nada (es el padre)
EmployeeList	Lista de empleados	Tabla con datos	empleados, loading, error, callbacks
EmployeeForm	Formulario	Campos de entrada	formData, errors, callbacks
EmployeeModal	Contenedor modal	Ventana emergente	isOpen, onClose, children, title

¿Por qué separarlos así?

BENEFICIOS DE COMPONENTIZAR:

- ✅ REUTILIZABILIDAD: EmployeeForm puede usarse para crear y editar
- ✅ MANTENIMIENTO: Cada componente es pequeño y fácil de modificar
- ✅ PRUEBAS: Se pueden probar de forma aislada
- ✅ ORGANIZACIÓN: Cada archivo tiene una sola responsabilidad
- ✅ LEGIBILIDAD: El código es más fácil de entender

Jerarquía de Componentes



Flujo de Datos entre Componentes

App (Padre)

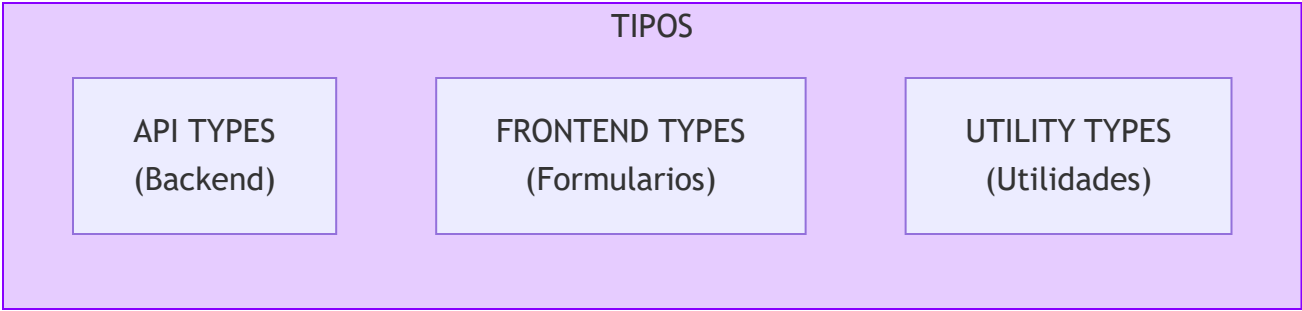
- Tiene: employees, loading, error
- Tiene: funciones handleCreate, handleEdit, handleDelete
- Le pasa a EmployeeList (Hijo):
 - employees (datos)
 - loading (estado)
 - error (estado)
 - onEdit (función)
 - onDelete (función)
 - onIncreaseSalary (función)
- Le pasa a EmployeeModal (Hijo):
 - isOpen (estado)
 - onClose (función)
 - children (EmployeeForm)
 - Le pasa a EmployeeForm:
 - formData (estado)
 - errors (estado)
 - isSubmitting (estado)
 - onChange (función)
 - onSubmit (función)
 - onCancel (función)

2. Tipos (El Modelo)

¿Qué son?

Los tipos definen la **forma de los datos** que maneja tu aplicación. Son como el "molde" que asegura que los datos tengan la estructura correcta.

Clasificación de Tipos



Tipos de API (Backend)

Tipo	Propósito	¿Dónde se usa?
Employee	Estructura de un empleado (respuesta del servidor)	EmployeeList, EmployeeAPI
CreateEmployeeRequest	Datos para crear (lo que envías al servidor)	EmployeeAPI.create()
UpdateEmployeeRequest	Datos para actualizar (lo que envías al servidor)	EmployeeAPI.update()
EmployeeListResponse	Respuesta del servidor al listar	EmployeeAPI.getAll()
EmployeeStatsResponse	Respuesta de estadísticas	EmployeeAPI.getStats()

Tipos de Frontend (Formularios)

Tipo	Propósito	¿Dónde se usa?
EmployeeFormState	Estado del formulario (string para inputs controlados)	App.tsx, EmployeeForm

Tipo	Propósito	¿Dónde se usa?
EmployeeFormErrors	Errores de validación del formulario	App.tsx, EmployeeForm

¿Por qué separar tipos de API y Frontend?

RAZONES:

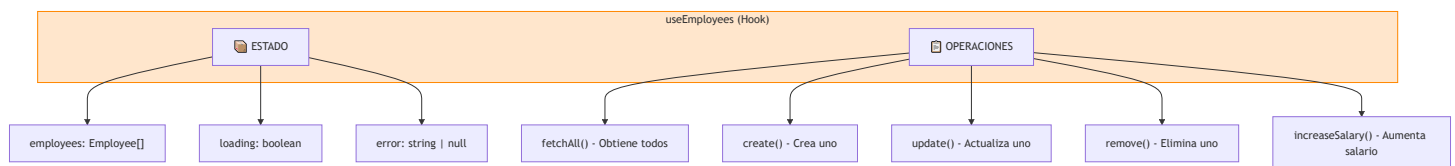
- ✅ INDEPENDENCIA: El backend puede cambiar sin afectar el frontend
- ✅ FLEXIBILIDAD: Puedes tener diferentes estructuras para diferentes usos
- ✅ SEGURIDAD: No expones datos internos del backend
- ✅ CLARIDAD: Sabes qué datos vienen de la API y cuáles son locales

⚙️ 3. Hooks (La Lógica de Negocio)

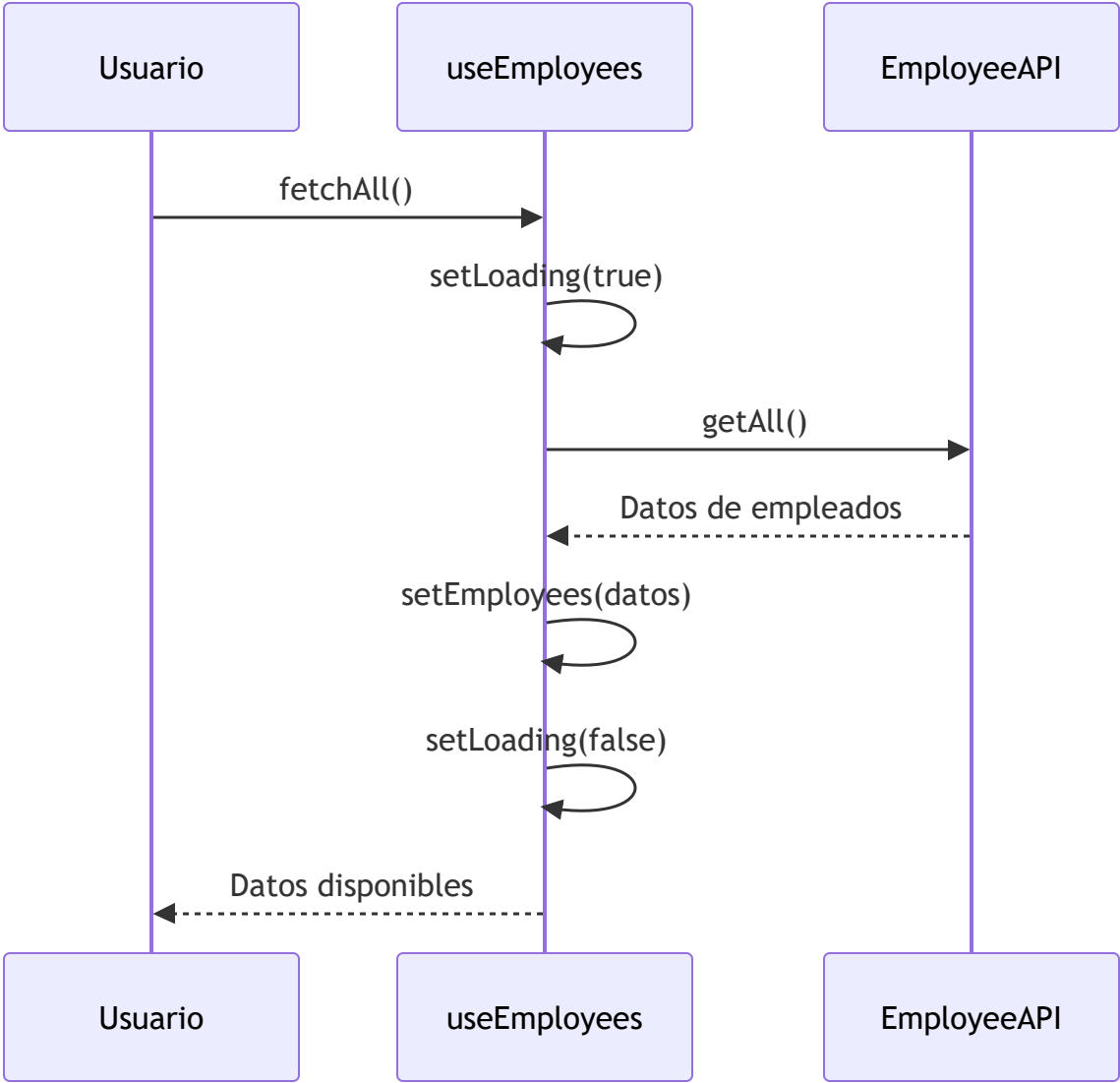
¿Qué son?

Los hooks son funciones que **encapsulan lógica reutilizable** y **manejan el estado** de tu aplicación. En React, un hook personalizado es como un "controlador" que centraliza la lógica.

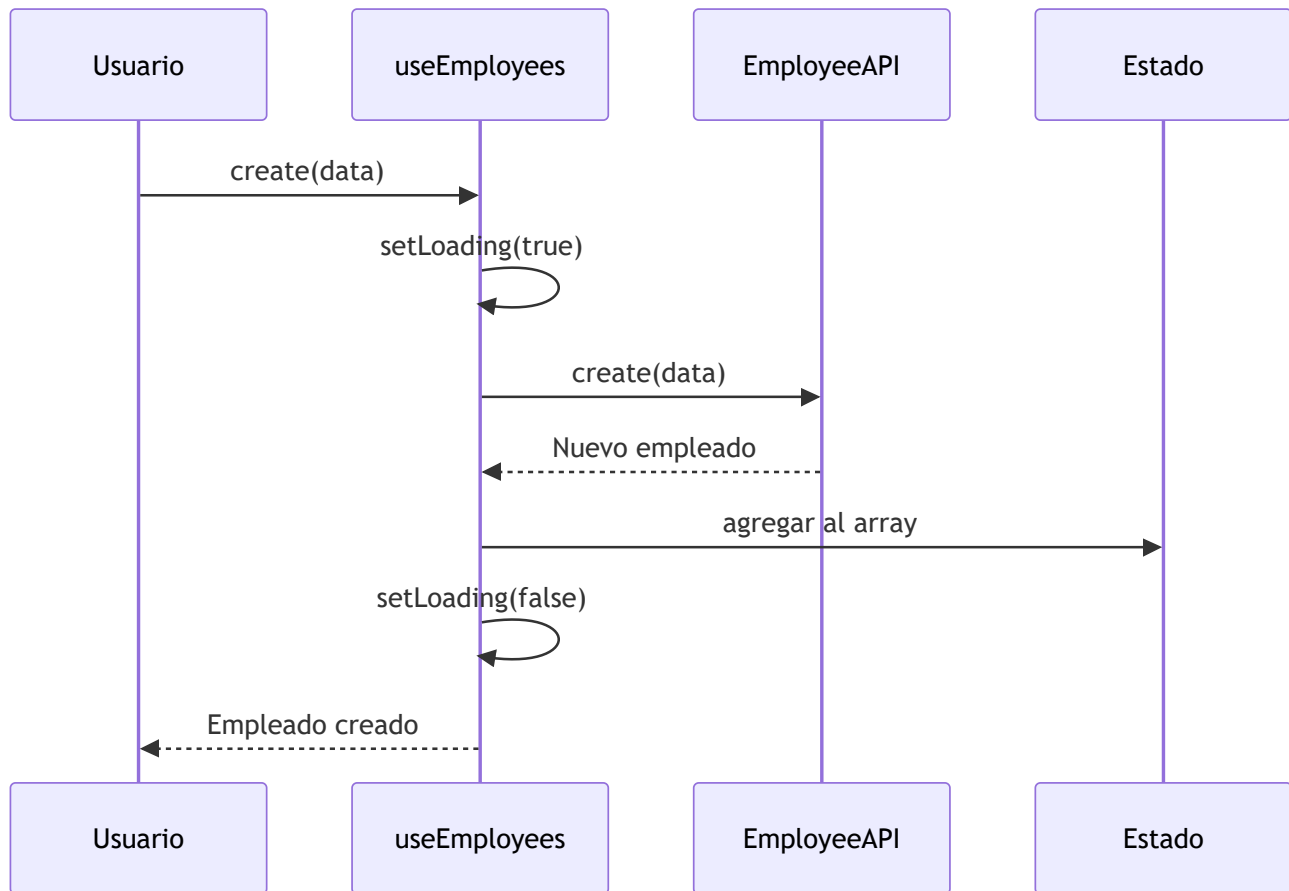
¿Qué hace useEmployees ?



Flujo de fetchAll



Flujo de create



¿Por qué usar un hook personalizado?

BENEFICIOS:

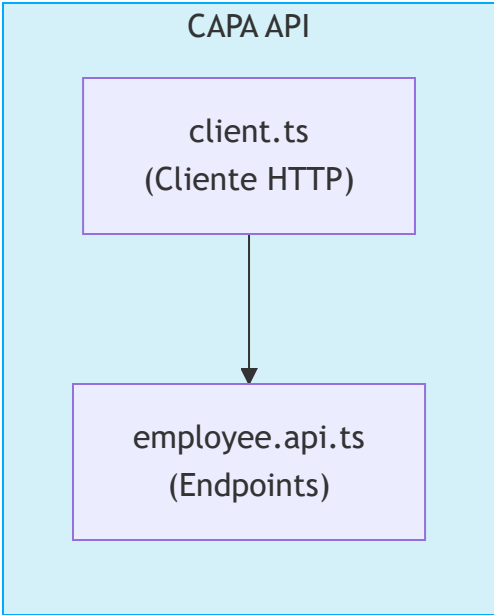
- ✅ REUTILIZACIÓN: La misma lógica se puede usar en varios componentes
- ✅ SEPARACIÓN: La lógica de negocio no está mezclada con la UI
- ✅ PRUEBAS: El hook se puede probar de forma aislada
- ✅ MANTENIMIENTO: Los cambios en la lógica se hacen en un solo lugar
- ✅ LEGIBILIDAD: El código de los componentes es más limpio

📢 4. API (La Capa de Datos)

¿Qué son?

La capa API es la **puerta de comunicación** entre tu frontend y el backend. Se encarga de hacer las peticiones HTTP y manejar las respuestas.

Estructura de la Capa API



client.ts - El Cliente HTTP

Elemento	Propósito
baseUrl	URL base del backend (desde .env)
headers	Cabeceras comunes (Content-Type)
timeout	Tiempo máximo de espera (10s)
interceptors	Logging de peticiones/respuestas

employee.api.ts - Los Endpoints

Método	Endpoint	Propósito
getAll()	GET /api/employees	Obtener todos los empleados

Método	Endpoint	Propósito
getById()	GET /api/employees/:id	Obtener un empleado
create()	POST /api/employees	Crear un empleado
update()	PUT /api/employees/:id	Actualizar un empleado
delete()	DELETE /api/employees/:id	Eliminar un empleado
increaseSalary()	PATCH /api/employees/:id/salary	Aumentar salario
getStats()	GET /api/employees/stats	Obtener estadísticas

¿Por qué separar API en dos archivos?

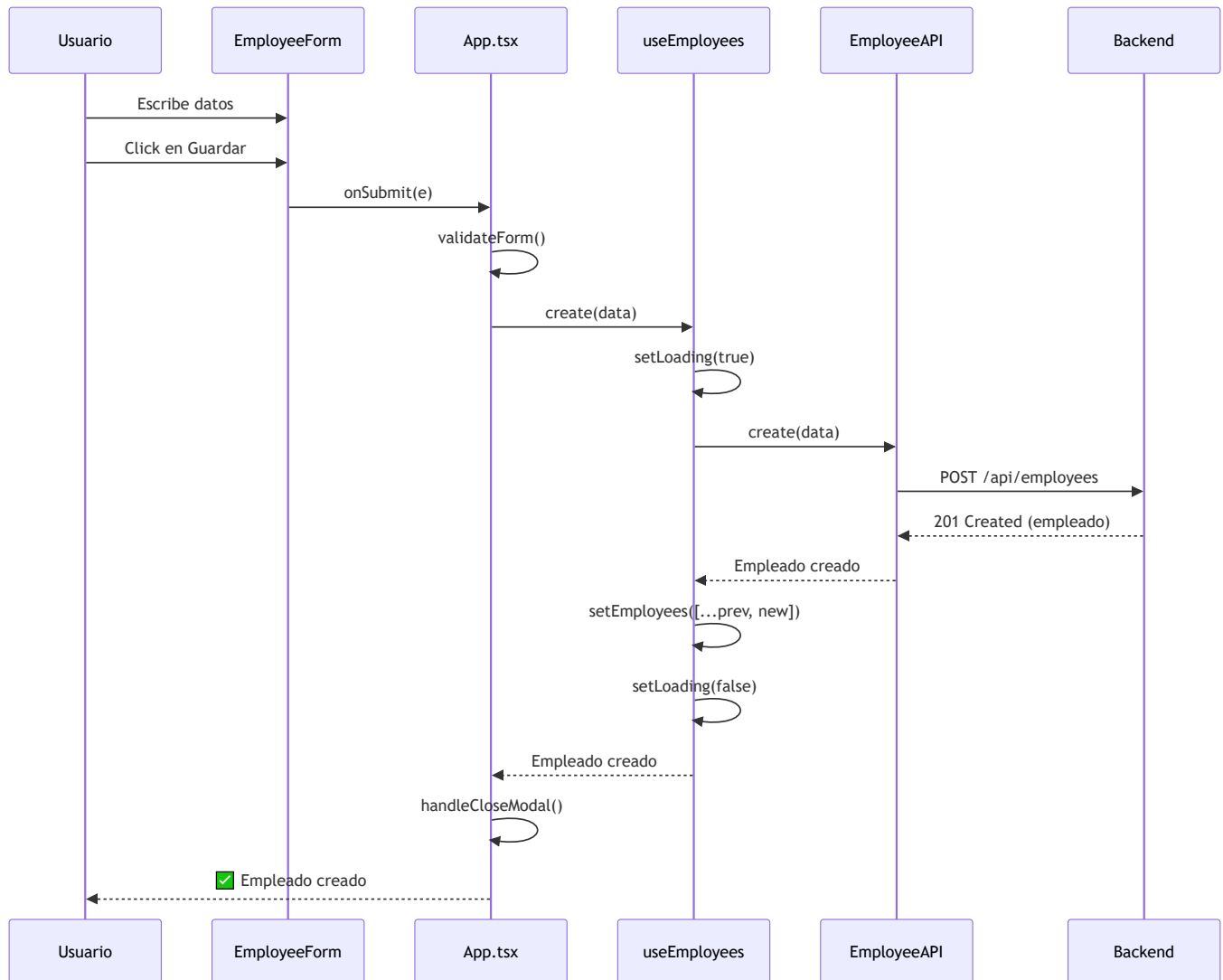
RAZONES:

- ✅ REUTILIZACIÓN: `client.ts` se puede usar para otras entidades
- ✅ ORGANIZACIÓN: Cada entidad tiene su propio archivo de endpoints
- ✅ MANTENIMIENTO: Los cambios en Axios se hacen en un solo lugar
- ✅ CONFIGURACIÓN: Interceptores, headers y timeout centralizados

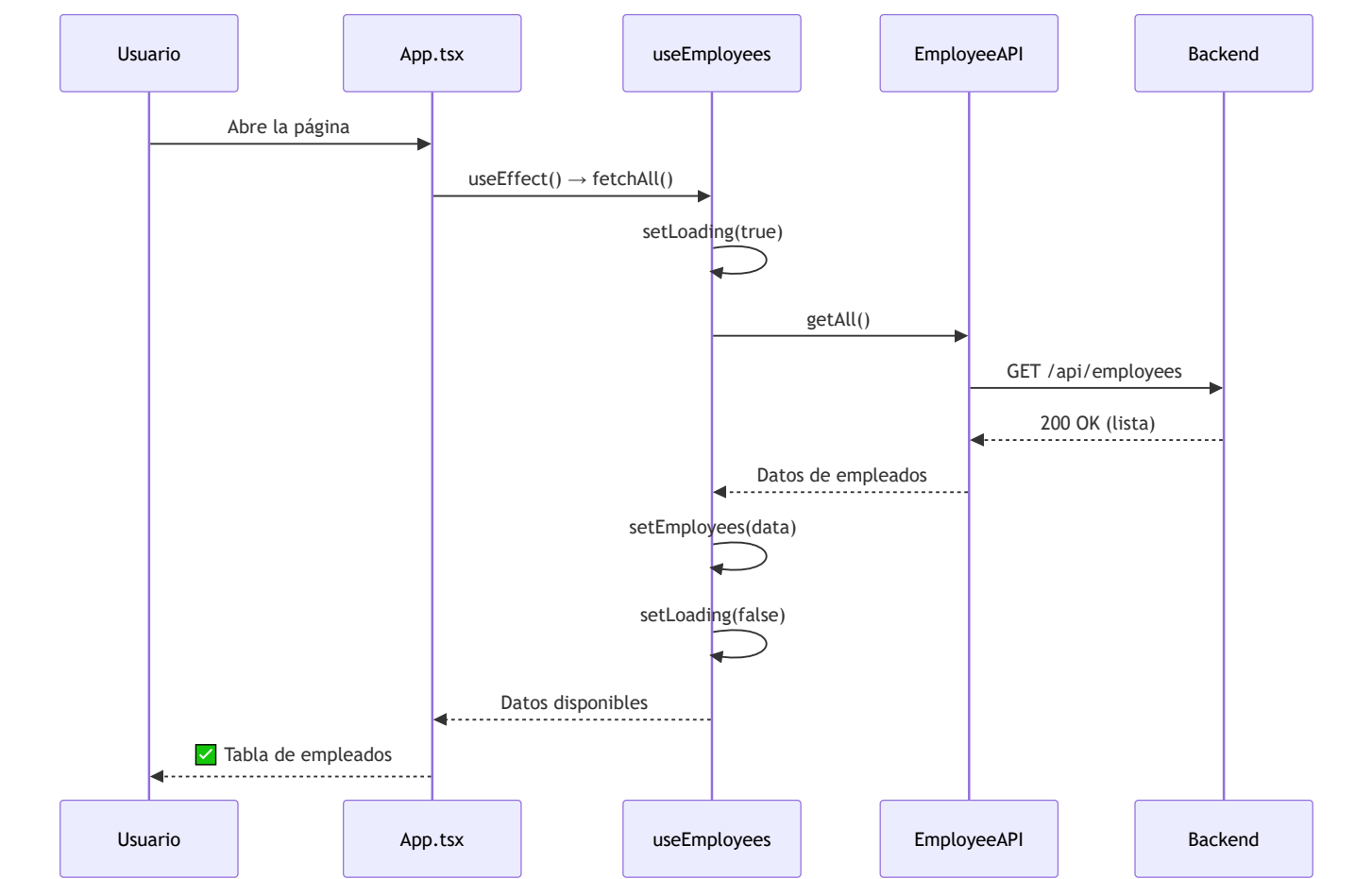


5. Flujo Completo de Datos

Crear un Empleado



Listar Empleados



6. Resumen de Responsabilidades

Capa	Archivo	Responsabilidad	¿Qué NO hace?
UI	App.tsx	Orquestrar componentes, manejar estado del formulario	No llama directamente a la API
UI	EmployeeList	Mostrar tabla de empleados	No modifica datos, solo recibe props
UI	EmployeeForm	Mostrar y capturar datos del formulario	No sabe qué hacer con los datos
UI	EmployeeModal	Mostrar/ocultar ventana modal	No contiene lógica de negocio

Capa	Archivo	Responsabilidad	¿Qué NO hace?
Lógica	useEmployees	Manejar estado y operaciones CRUD	No sabe cómo se muestran los datos
API	employee.api.ts	Definir endpoints y hacer peticiones	No contiene lógica de negocio
API	client.ts	Configurar Axios (baseURL, headers, interceptors)	No sabe qué endpoints existen
Tipos	employee.types.ts	Definir interfaces de datos	No contiene lógica ni valores

7. Checklist de Verificación

ENTENDISTE...

- ☐ ¿Qué es un componente y por qué separarlos?
- ☐ ¿Cuántos componentes tienes y qué hace cada uno?
- ☐ ¿Qué son los tipos y por qué separarlos en API y Frontend?
- ☐ ¿Qué hace el hook useEmployees y por qué es importante?
- ☐ ¿Cómo se comunica el frontend con el backend?
- ☐ ¿Cuál es el flujo de datos desde el usuario hasta el backend?
- ☐ ¿Por qué está organizada la estructura de esta manera?



8. Analogía Final

IMAGINA QUE ESTE PROYECTO ES UN RESTAURANTE:



EMPLOYEELIST = La carta del menú

→ Muestra lo que hay disponible (los empleados)



EMPLOYEEFORM = La comanda (pedido)

→ Captura lo que el cliente quiere (nuevo empleado)



EMPLOYEEMODAL = La puerta de la cocina

→ Abre/cierra el acceso al área de preparación



APP = El gerente del restaurante

→ Coordina todo: recibe pedidos, los envía a cocina, sirve la mesa



USEEMPLOYEES = El chef

→ Prepara los platos (procesa los datos)



EMPLOYEE.API = El proveedor de ingredientes

→ Trae lo que necesita el chef desde el almacén (backend)



EMPLOYEE.TYPES = La receta

→ Define cómo debe ser cada plato (la estructura de los datos)